

Final Term Project Report — Advanced Programming

Student Name: Ali Asghar Rashidabadi

Student ID: 40419533

Field of Study: Computer Science **Degree:** Bachelor's

Course: Advanced Programming — Final Term Project Report

Instructor: Dr. Ahmadi

The project consists of 8 files. Each file is responsible for a separate task, and other files import the relevant functions or classes from it as needed.

1. Utils.py

This file contains the general-purpose functions needed throughout the program — functions for validation and sorting based on user input. Checking whether input is capitalized, whether input is numeric, and similar checks are the general functions in this file.

The sorting algorithm used in this file is based on merge sort, which allows the user to view student transcript data sorted according to a chosen key (sort reference). Because this algorithm halves the list on each pass of the loop and then performs one traversal over the length of the list, it has a time complexity of $\Theta(n \log n)$.

2. Major.py

The class that manages fields of study (majors). This class stores the fields of study entered by the user, after the necessary validation, so that other classes can access them.

3. Person.py

Parent class for the Student class. This class assigns personal information such as first name, last name, and national ID to the relevant attributes after the necessary validation.

4. Student.py

Child class of Person. This class assigns student information such as student ID, degree, major, term, course codes, and course grades to the relevant attributes after the necessary validation.

This class primarily controls student information: calculating the student's GPA, building a dictionary of course codes and grades, adding/removing a student's courses, checking whether a student has passed a course/term, and viewing the student's transcript. It also includes dunder-methods for checking equality and comparing whether one student's GPA is greater or less than another's.

5. Subject.py

This class is responsible for creating the courses offered during the term. A student can only select course codes that have been created in this class. The course code, course name, and course units are assigned to the relevant attributes after the necessary validation. It also includes dunder-methods for comparing whether one course's units are greater or less than another's.

6. EduSysManager.py

The class that manages the educational system. Using the methods of the other classes, this class controls the information of all students: adding/removing a student to/from the system, adding/removing a course, finding a specific student, and sorting students.

7. Analytics.py

This class is responsible for analyzing student grades. After the user specifies a basis (degree level, term, etc.) and a key (a specific value for that basis — e.g., Bachelor's as the degree level, 3 as the term), the

program calculates the highest and lowest scores, the mean and standard deviation of the scores, and the number of passes and fails.

8. Main

The Main section of the program, which is the primary execution environment. The user drives the program forward by choosing from the available commands (functions stored in a dictionary), proceeds according to the instructions, and exits the program whenever desired.

Sorting Algorithm Used and Time Complexity

The sorting algorithm used is based on merge sort. This algorithm first receives a basis from the user (GPA, first name, last name) to sort by, and sorts the students according to this basis.

Because this algorithm halves the length of the list on each iteration, it has a time complexity of $\theta(\log n)$. In addition, because there is another traversal over all elements of the list, of length $\text{len}(l)$, the time complexity of that part is $\theta(\text{len}(l))$, or $\theta(n)$ for short, where $n = \text{len}(l)$. Therefore, the overall time complexity of this algorithm is:

$$\Theta(\log n) \times \theta(n) = \theta(n \log n)$$

Statistical Analysis Performed

The statistical analysis is performed using the numpy library. First, a basis (all, degree level, major, term, course) and a key (the value specified for that basis) are received from the user. Then, statistical analyses are carried out for the students who satisfy the condition specified by the key.

The analyses are carried out with the numpy library. The scores that satisfy the key's condition are stored in a list, which is converted into an array and passed to the next method. There, the highest and lowest scores, the mean and standard deviation of the scores, and the number of passes and fails are calculated.
